

Verilog 第 4 次实验报告

学生姓名	王一诺	学号	2312331	指导老师	董前琨老师
------	-----	----	---------	------	-------

一、实验内容及步骤

1. 请自行设计一个模块，完成统计 32 位 2 进制数 0 和 1 出现的次数。

测试文件：

```
module tb();
    reg [31:0] data_in; // 测试数据输入
    wire [5:0] count_0; // 0的计数输出
    wire [5:0] count_1; // 1的计数输出
    // 实例化待测试的模块
    count_bits uut (
        .data_in(data_in),
        .count_0(count_0),
        .count_1(count_1)
    );
    // 生成32位随机数
    initial begin
        // 生成随机数的时间间隔，可以根据需要调整
        #1 data_in = $random; // 初始随机数
        forever begin
            #10 data_in = $random; // 每隔10个时间单位生成一个32位随机数
        end
    end
    // 监控输出
    initial begin
        $monitor("At time %t, data_in = %h, count_0 = %d, count_1 = %d",
            $time, data_in, count_0, count_1);
    end
    // 测试结束时间
    initial begin
        #1000 $finish; // 运行1000个时间单位后结束仿真
    end
endmodule
```

模块设计：

```
module count_bits(
    input wire [31:0] data_in, // 32位输入数据
    output reg [5:0] count_0, // 统计0的个数
    output reg [5:0] count_1 // 统计1的个数
);
    integer i;

    always @(*) begin
        count_0 = 0;
        count_1 = 0;
        for (i = 0; i < 32; i = i + 1) begin
            if (data_in[i] == 1'b0)
                count_0 = count_0 + 1; // 统计0的数量
            else
                count_1 = count_1 + 1; // 统计1的数量
        end
    end
endmodule
```

测试样例：

Name	Value	999,997 ps	999,998 ps	999,999 ps
> data_in[31:0]	bc148878		bc148878	
count_0[5:0]	13		13	
count_0[5]	0		0	
count_0[4]	1		1	
count_0[3]	0		0	
count_0[2]	0		0	
count_0[1]	1		1	
count_0[0]	1		1	
count_1[5:0]	0d		0d	
count_1[5]	0		0	
count_1[4]	0		0	
count_1[3]	1		1	
count_1[2]	1		1	
count_1[1]	0		0	
count_1[0]	1		1	

图中所在时刻 data_in 实际为
1011110000010100100010000111
1000（过长所以图中未展开每一位），输出 count_0=010011 即 19
个 0, count_1=001101 即 13 个 1，
验证正确。

2. 请使用 always 块语句，实现一个十分频器

模块设计：

测试文件：

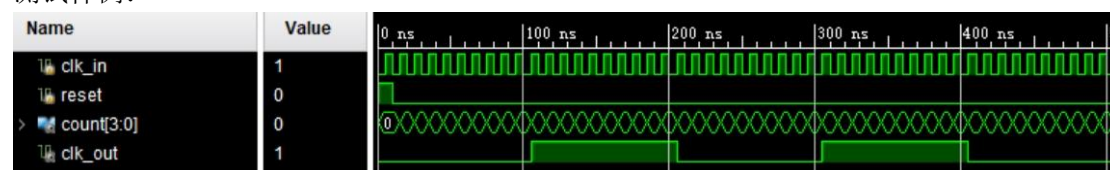
```

module divider10 (
    input clk_in,    // 输入时钟信号
    input reset,     // 重置信号
    output reg [3:0] count, // 计数器 (0~9)
    output reg clk_out // 输出时钟信号
);
    always @(posedge clk_in or posedge reset) begin
        if (reset) begin
            count <= 0; // 计数器清零
            clk_out <= 0; // 输出时钟初始为0
        end else begin
            if (count == 9) begin
                count <= 0; // 当计数器到9时清零
                clk_out <= ~clk_out; // 翻转时钟
            end else begin
                count <= count + 1; // 计数器加1
            end
        end
    end
endmodule

module test_divider10;
    reg clk_in;
    reg reset;
    wire [3:0] count;
    wire clk_out;
    // 实例化十分频器模块
    divider10 out (
        .clk_in(clk_in),
        .reset(reset),
        .count(count),
        .clk_out(clk_out)
    );
    // 生成测试输入时钟信号
    initial begin
        clk_in = 0;
        forever #5 clk_in = ~clk_in;
    end
    // 测试用例
    initial begin
        reset = 1; // 初始化复位
        #10 reset = 0;
        #500 $stop; // 等待一段时间结束仿真
    end
endmodule

```

测试样例：



二、总结

1. 基础知识的巩固：通过实现这么一个简单的分频器，可以加深对数字电路设计基本概念的理解，例如时钟信号、计数器、触发器等。应用 verilog 可以帮助我巩固数字逻辑的知识，而巩固的数字逻辑知识利于我更好地设计模块。再加上数字逻辑实验课也做过分频，切实感受到了啥叫理论实践结合。

2. 感受到了系统思维与细节化关注结合的重要性。设计数字系统时，不仅需要从整体上考虑系统的工作流程和状态转换，也要注意到了代码细节，比如变量定义、信号赋值、对时序的精确控制等。有纵览大局的设计思维，有细嗅蔷薇的创作专注力，这不仅是数字逻辑设计的诀窍，也是一切学习和创作的至臻境界。